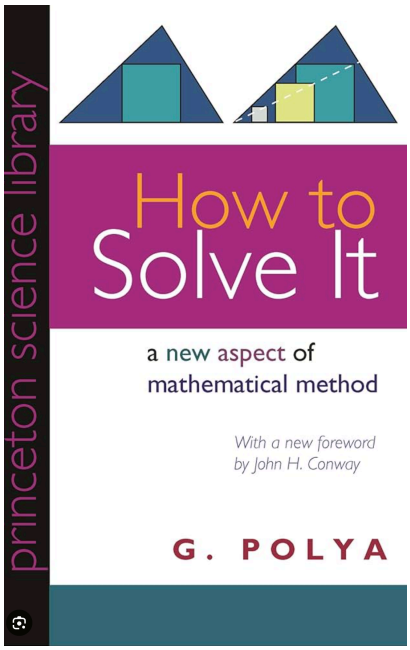
Polya



uPCB

Understand : #2259

github.com/google-deepmind/mujoco/issues/2259

google-deepmind / mujoco

Type / to search

Issues 154 | Pull requests 181 | Agents | Discussions | Actions | Security and quality | Insights

[MJX] jax.lax.while_loop in solver.py prevents computation of backward gradients #2259

New issue

Open | #3264 (+2)

EGalahad opened on Nov 30, 2024

The feature, motivation and pitch

Problem

The solver's `jax.lax.while_loop` implementation prevents gradient computation through the environment step during gradient based trajectory optimization. This occurs in the solver implementation when iterations > 1.

Error encountered with `jax.jit` compiled grad function:

```
ValueError: Reverse-mode differentiation does not work for lax.while_loop or lax.fori_loop with dynamic sta
```

Current workaround of using `opt.iteration=1` leads to potentially inaccurate simulation and gradients.

Proposed Solution

Add an option to set a fixed iteration count (e.g., 4) that would be compatible with reverse-mode differentiation using either `lax.scan` or `lax.fori_loop` with static bounds.

Alternatives

No response

Additional context

No response

5

EGalahad added enhancement on Nov 30, 2024

saran-t assigned erikfrey and btaba on Dec 4, 2024

erikfrey added good first issue on Dec 4, 2024

Assignees: btaba, erikfrey

Labels: MJX, enhancement, good first issue

Type: No type

Projects: No projects

Milestone: No milestone

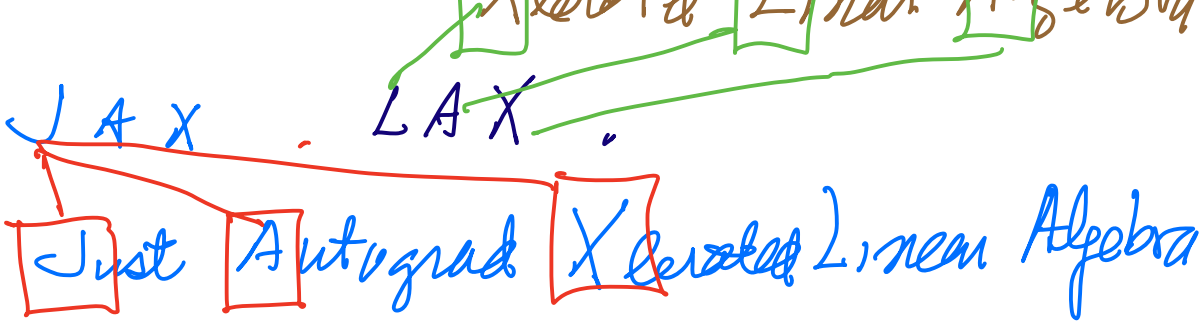
Relationships: None yet

Development:
- feat(mjx): Make solver differentiable and fix name collisions — google-deepmind/mujoco
- [MJX] Fix gradient computation by replacing jax.lax.while_loop with _while_loop_scan — theshloksschauhan/mujoco
- [MJX] Use scan-based loop in solver to enable reverse-mode autodiff (#2259) — google-deepmind/mujoco

Notifications: Customize — Subscribe

You're not receiving notifications from this thread.

Identify Ways Algebra

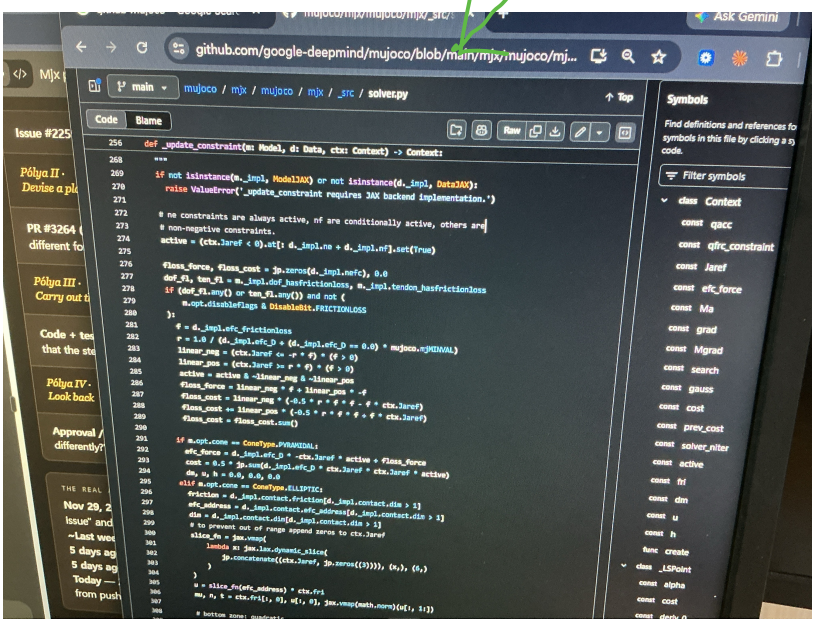

JAX
LAX
Just
Autograd
Linear Algebra

Plan : include/mujoco

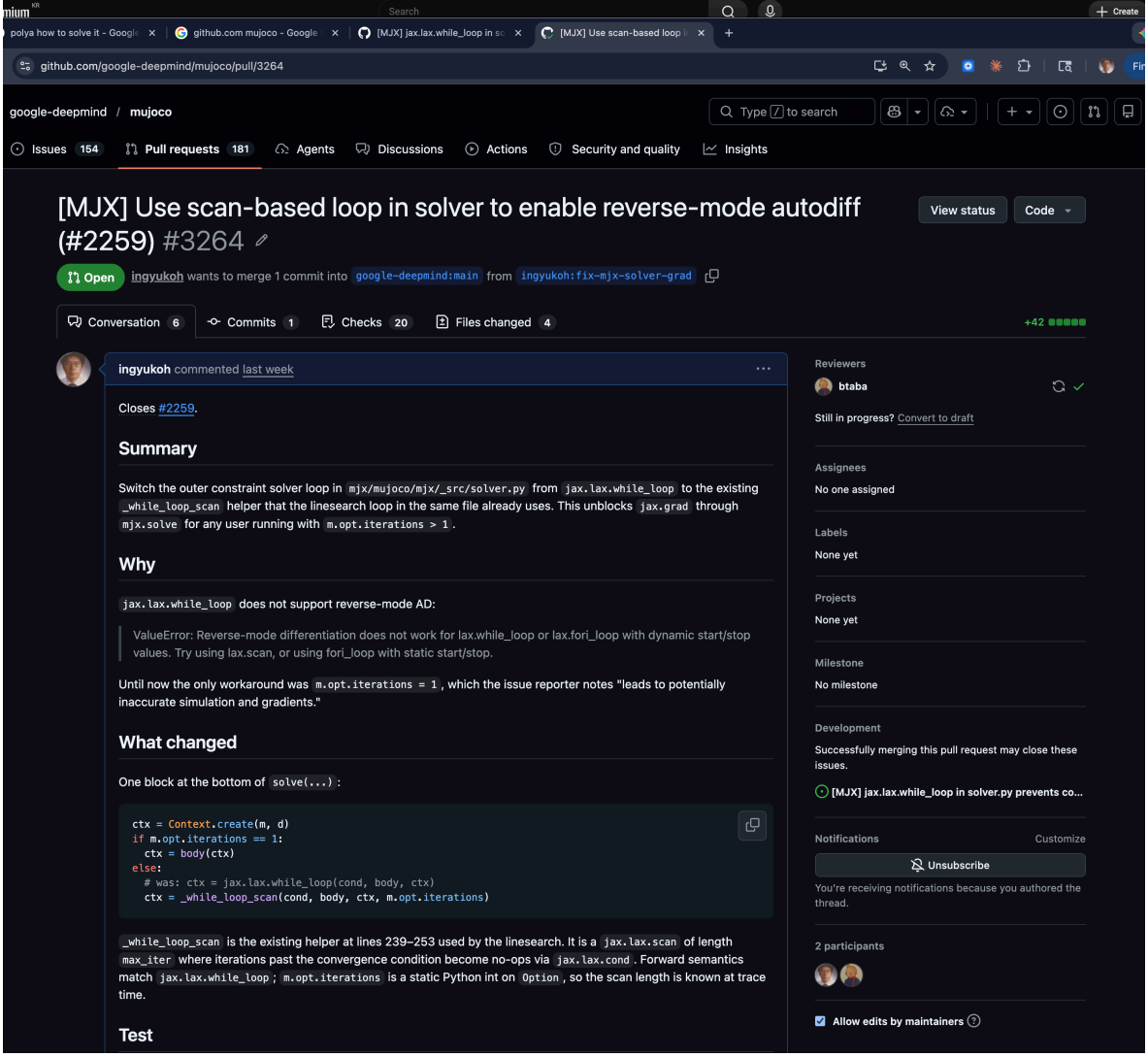
mjx

model

src



Carry Out : #3264

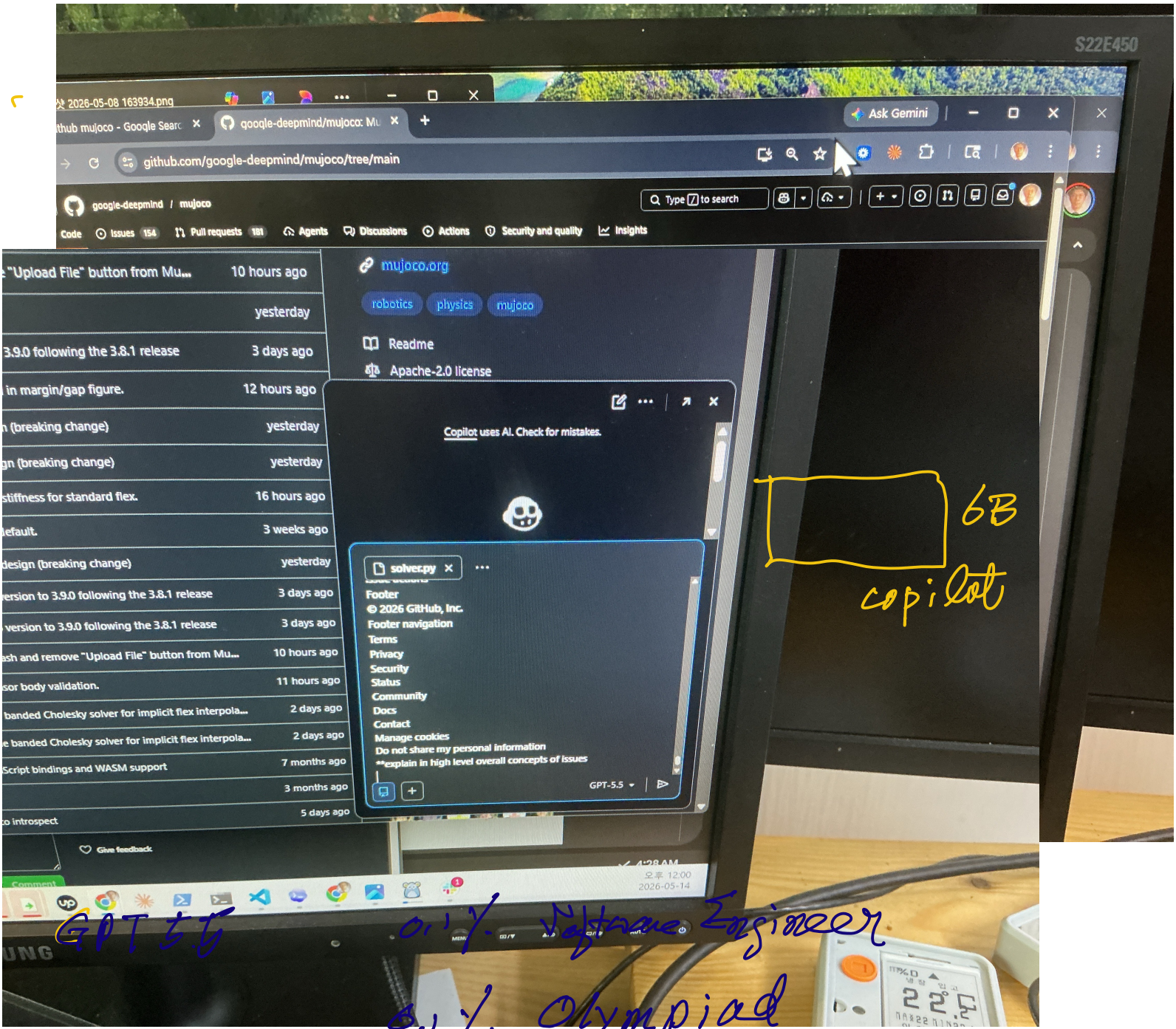


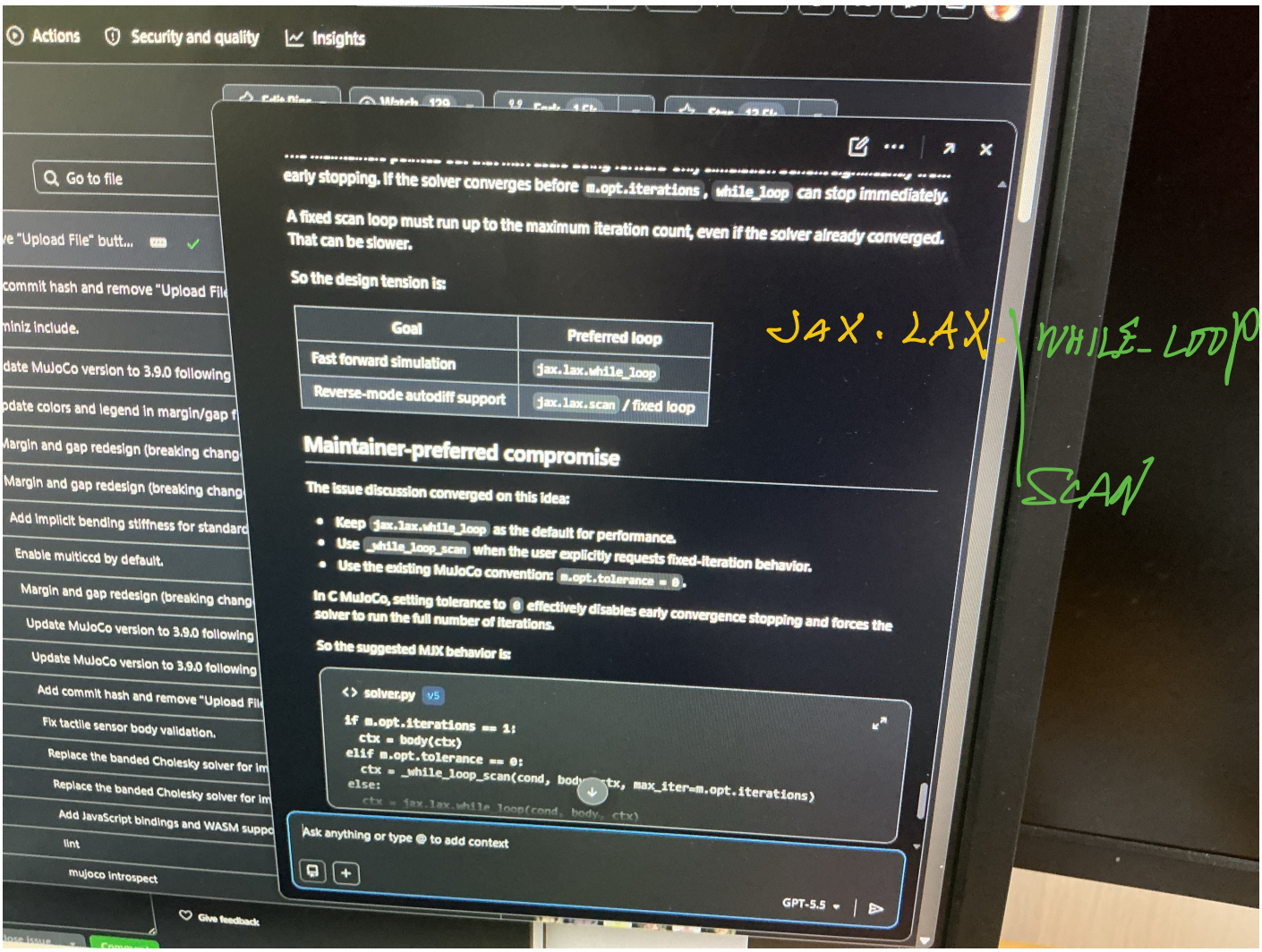
Back :

btata : reviewer

ask ||

solve


S22E450
2026-05-08 163934.png
google-deepmind/mujoco: Mu
Ask Gemini
github.com/google-deepmind/mujoco/tree/main
google-deepmind / mujoco
Type / to search
Code
Issues 154
Pull requests 181
Agents
Discussions
Actions
Security and quality
Insights
"Upload File" button from Mu... 10 hours ago
yesterday
3.9.0 following the 3.8.1 release 3 days ago
in margin/gap figure. 12 hours ago
(breaking change) yesterday
gn (breaking change) yesterday
stiffness for standard flex. 16 hours ago
default. 3 weeks ago
design (breaking change) yesterday
ersion to 3.9.0 following the 3.8.1 release 3 days ago
version to 3.9.0 following the 3.8.1 release 3 days ago
ash and remove "Upload File" button from Mu... 10 hours ago
sor body validation. 11 hours ago
banded Cholesky solver for implicit flex interpola... 2 days ago
e banded Cholesky solver for implicit flex interpola... 2 days ago
Script bindings and WASM support 7 months ago
3 months ago
5 days ago
o introspect
Give feedback
mujoco.org
robotics
physics
mujoco
Readme
Apache-2.0 license
Copilot uses AI. Check for mistakes.
solver.py
Footer
© 2026 GitHub, Inc.
Footer navigation
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information
**explain in high level overall concepts of issues
GPT-5.5
6B
copilot
GPT 5.5
0.1%. Software Engineer
0.1%. Olympiad


Actions
Security and quality
Insights
Go to file
early stopping. If the solver converges before m.opt.iterations, while_loop can stop immediately.
A fixed scan loop must run up to the maximum iteration count, even if the solver already converged. That can be slower.
So the design tension is:
Goal
Preferred loop
Fast forward simulation
jax.lax.while_loop
Reverse-mode autodiff support
jax.lax.scan / fixed loop
Maintainer-preferred compromise
The issue discussion converged on this idea:
Keep jax.lax.while_loop as the default for performance.
Use _while_loop_scan when the user explicitly requests fixed-iteration behavior.
Use the existing MuJoCo convention: m.opt.tolerance = 0.
In C MuJoCo, setting tolerance to 0 effectively disables early convergence stopping and forces the solver to run the full number of iterations.
So the suggested MJX behavior is:
solver.py
v5
if m.opt.iterations == 1:
ctx = body(ctx)
elif m.opt.tolerance == 0:
ctx = _while_loop_scan(cond, body, ctx, max_iter=m.opt.iterations)
else:
ctx = jax.lax.while_loop(cond, body, ctx)
Ask anything or type @ to add context
GPT-5.5
Give feedback
JAX . LAX | WHILE_LOOP
SCAN